

Documentation technique

Vite & Gourmand

Application web de commande en ligne de menu traiteur

Projet	Vite & Gourmand – Application web et mobile
Client	Julie & José – Vite & Gourmand, Bordeaux
Développeur	FastDev
Stack	HTML5/SCSS/JS/PHP/MySQL/MongoDB/Heroku
Date	2026

Table des matières

Vite & Gourmand	0
1. Réflexions initiales technologiques	3
1.1. Contexte.....	3
1.2. Choix technologique	3
2. Architecture Technique	4
3. Environnement de développement	4
3.1. Environnement de développement local	4
3.2. Extensions VS Code utilisées.....	5
3.3. Services Docker.....	5
4. Modèle conceptuel de données (MCD)	6
4.1. Modèle relationnel (MySQL)	7
a. Entités principales.....	7
b. Relations.....	7
c. Vues SQL	8
4.2. Modèle NoSQL – collections MongoDB	9
a. Collection avis	9
b. Collection commandes.....	9
c. Collection demandesClient.....	10
5. Diagrammes d'utilisation	11
5.1. Diagramme de cas d'utilisation (Use Case)	11
5.2. Diagramme de séquence.....	13
a. Commande d'un menu.....	13
b. Commande terminée – Avis client	14
6. Déploiement de l'application	14
6.1. Prérequis	14
6.2. Architecture de déploiement.....	15
6.3. Structure du projet.....	15
6.4. Fichiers de configuration Heroku	15
a. Procfile.....	15
b. config/nginx.conf.erb	15
c. composer.json	16
6.5. Déploiement.....	16
a. Étape 1 - Création de l'application Heroku	16
b. Étape 2 - Configuration des variables d'environnement.....	16
c. Étape 3 - Déploiement	17
6.6. Base de données	17
a. MySQL (JawsDB)	17
b. MongoDB Atlas	17
c. Cloudinary	17
6.7. Vite-et-gourmand.fr, DSN et sécurité	17
a. Délégation DNS vers Cloudflare	17
b. Configuration DNS.....	18
c. Configuration Cloudflare	18
d. DNSSEC :	18

6.8.	Variables d'environnement.....	19
6.9.	Vérification du déploiement	19
6.10.	Problèmes rencontrés et solutions.....	19

1. Réflexions initiales technologiques

1.1. Contexte

Le projet consiste à développer une application web et mobile complète pour l'entreprise Vite & Gourmand, permettant la consultation de menus traiteurs, la commande en ligne, et la gestion administrative par les employés et l'administrateur. Le client souhaite augmenter sa visibilité et simplifier la prise de commande, jusqu'alors réalisée uniquement par mail ou par téléphone. Le projet a été développé en individuel dans le cadre du TP Studi – Développeur web et mobile. L'application cible deux types d'utilisateurs : les clients (commande en ligne) et les employés/administrateur (gestion back-office). Une attention particulière a été portée à la sécurité (JWT, Bcrypt) et à l'accessibilité (RGAA).

1.2. Choix technologique

Front-end

Technologie	Justification
HTML5	Standard web, sémantique, accessibilité RGAA
SCSS	Sur-ensemble de css : variables, code maintenable et structuré
Bootstrap 5	Framework CSS : utilisé pour un développement rapide et responsif
JavaScript vanilla	Pour prise en main JS, manipulation du DOM, filtrage dynamique, ...

Back-end

Technologie	Justification
PHP (PDO)	Rapide, grande communauté, aide à maîtriser les requêtes SQL, léger et adapté aux API REST
JWT (JSON WEB TOKEN)	Gestion des rôles, authentification sécurisée
PHP Mailer	Envoi de mails automatiques : bienvenue, confirmation commande, réinitialisation mot de passe, avis
Bcrypt	Hashage sécurisé des mots de passe avant stockage en base de données

Base de données

Technologie	Justification
MySQL	Base de données relationnelle robuste, gestion des relations complexes (menus, plats, commandes, user, rôles, régimes, ...)
MongoDB	Base NoSQL utilisée pour les statistiques de commandes par menu, avis — données agrégées pour les graphiques du dashboard admin, synchronisées depuis MySQL

Déploiement

Technologie	Justification
Heroku	Simple à configurer, déploiement via git, add-ons MySQL disponible et simple à intégrer

2. Architecture Technique

L'application **Vite & Gourmand** repose sur une architecture **client-serveur découplée**, conteneurisée via **Docker Compose** pour l'environnement de développement et déployée sur **Heroku** en production.

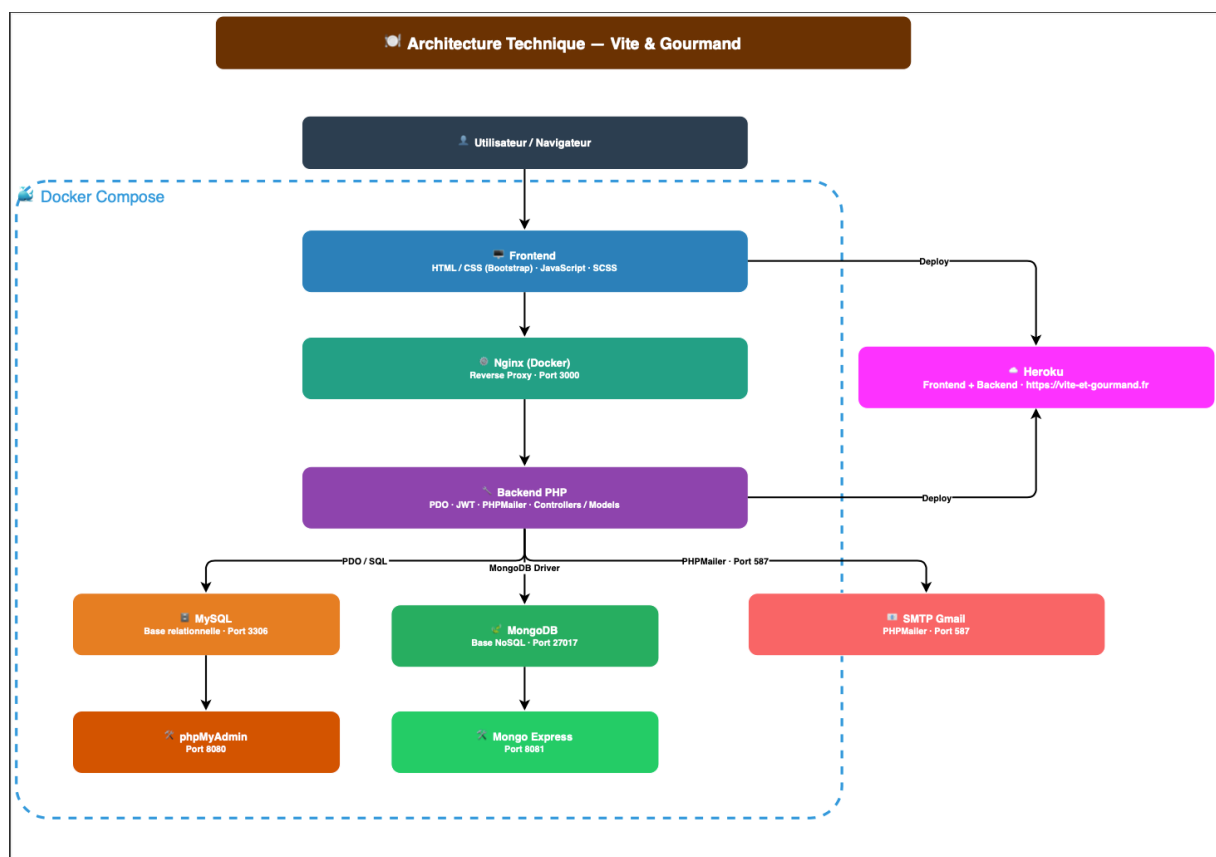


Figure 1: [Architecture technique](#)

3. Environnement de développement

3.1. Environnement de développement local

Le projet tourne sous Docker, évitant toute installation manuelle de PHP ou MySQL en local. VS Code est utilisé comme éditeur principal avec les extensions listées ci-dessous.

Un fichier `.env` est requis à la racine du projet. Un fichier `.env` exemple est fourni dans le dépôt.

Outil	Version	Rôle
VS Code	Dernière version stable	Éditeur de code principal
Docker + Docker Compose	Dernière version stable	Conteneurisation MySQL, MongoDB, PHP, Nginx
Git	Dernière version stable	Versioning du code
GitHub		Hébergement du dépôt distant
Composer	2.x	Gestionnaire de dépendances PHP
Node JS	Dernière version stable	Nécessaire l'installation de Bootstrap

3.2. Extensions VS Code utilisées

Extension	Utilité
PHP Intelephense	Autocomplétions PHP
PHP Server	Servir le front en local sur le port 3000
SCSS IntelliSense	Autocomplétions SCSS
Database	Administrer la BDD MySQL depuis Vs code
Live Sass Compiler	Compiler SCSS vers CSS
MongoDB for Vs code	Se connecter à MongoDB et Atlas

3.3. Services Docker

Un docker-compose up -d suffit à démarrer l'ensemble des services. Le détail complet est disponible dans le README.md du dépôt GitHub.

Service	Image	Rôle
Nginx	nginx :1.27-alpine	Serveur web / reverse proxy
PHP	Php-fpm	Serveur PHP
MySQL	mysql :8.0	Base de données relationnelle
PhpMyAdmin	phpmyadmin :5.2-apache	Interface graphique MySQL
MongoDB	mongo :7.0	Base de données NoSQL
Mongo-express	mongo :1.0	Interface graphique MongoDB

Toutes les étapes de configuration du projet sur l'environnement de travail local sont décrites dans le fichier README.MD disponible de dépôt git.

4. Modèle conceptuel de données (MCD)

La base de données relationnelle repose sur plusieurs entités principales organisées autour du menu et de la commande.

L'entité users centralise tous les profils (clients, employés, admins) via une relation avec la table roles. Chaque utilisateur peut passer plusieurs commandes, chacune liée à un menu et dotée d'un statut courant. L'historique complet des changements de statut est conservé dans la table commande_historique_statuts, permettant une traçabilité totale.

L'entité menus est au cœur du modèle : un menu est composé de plusieurs plats (relation N:N), peut être associé à un ou plusieurs thèmes (Noël, Pâques, Classique, événement...) et à un ou plusieurs régimes alimentaires (Végétarien, Végan, Classique, Hallal, Sans gluten...). Chaque menu peut également disposer de plusieurs images stockées via Cloudinary.

Les plats sont typés (entrée, plat, dessert) et peuvent être liés à plusieurs allergènes, permettant d'informer les clients sur les risques alimentaires.

Enfin, les horaires d'ouverture sont gérés jour par jour grâce à une relation avec la table jours, et sont modifiables depuis l'espace employé.

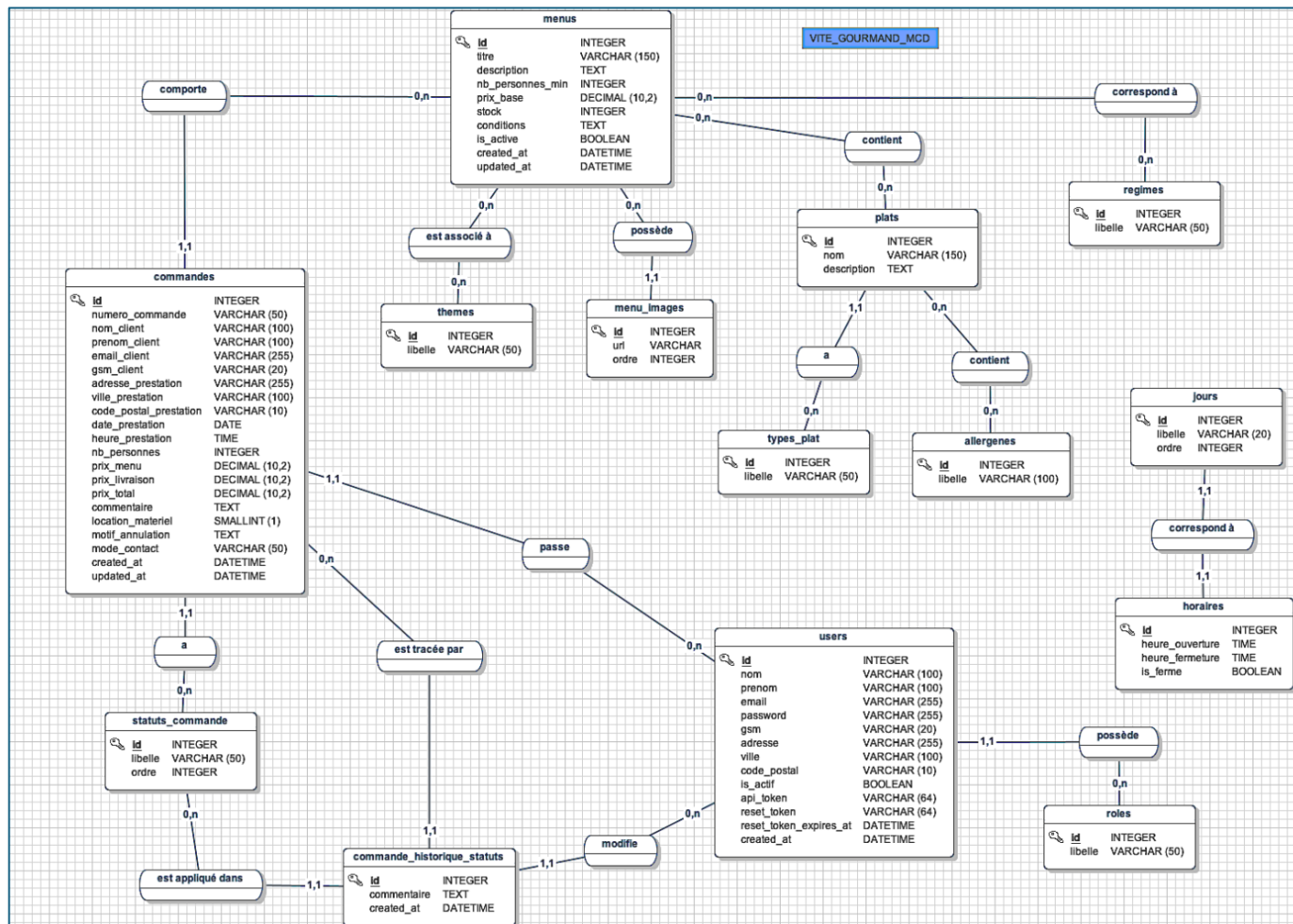


Figure 2: Modèle conceptuel de données (MCD)

4.1. Modèle relationnel (MySQL)

a. Entités principales

La base de données relationnelle est composée de ces 14 tables qui seront liées entre elle par des relations entraînant dans certains cas des tables pivot.

Entité	Description
users	Clients, employés et admins de la plateforme
menus	Menus proposés par le traiteur
plats	Plats composant les menus
types_plat	Liste des types de plat : Entrée, Plat et Dessert
commandes	Commandes passées par les clients
horaires	Horaires d'ouverture du restaurant
allergenes	Allergènes associés aux plats
themes	Thèmes associés aux menus (Noël, Pâques...)
regimes	Régimes alimentaires (vegan, hallal...)
jours	Les jours de la semaine avec un ordre déterminé : (Lundi, 1) > (Dimanche, 7)
roles	Différents rôles liés à un user (admin, employe, utilisateur)
Status_commande	Liste des différents statuts d'une commande : en_attente, accepte, en_preparation, en_cours_livraison, livre, en_retour_materiel, terminee, annulee
menu_images	Table de stockage des images d'un menu

b. Relations

Relation	Cardinalité	Description
users → roles	N:1	Un user a exactement 1 rôle. Un rôle peut être attribué à 0 ou plusieurs users
commandes → users	N:1	Une commande est passée par exactement 1 user. Un user peut passer 0 ou plusieurs commandes
commandes → menus	N:1	Un menu peut être dans 0 ou plusieurs commandes. Une commande concerne exactement 1 menu
commandes → statuts_commande	N:1	Une commande a exactement 1 statut courant. Un statut peut s'appliquer à 0 ou plusieurs commandes
commande_historique_statuts → commandes	N:1	Une commande peut avoir 0 ou plusieurs entrées d'historique. Un historique appartient à exactement 1 commande

menus ↔ plats	N:N	Un menu peut contenir 0 ou plusieurs plats. Un plat peut figurer dans 0 ou plusieurs menus
menus ↔ themes	N:N	Un menu peut avoir 0 ou plusieurs thèmes. Un thème peut correspondre à 0 ou plusieurs menus
menus ↔ regimes	N:N	Un menu peut correspondre à 0 ou plusieurs régimes. Un régime peut s'appliquer à 0 ou plusieurs menus
plats ↔ allergenes	N:N	Un plat peut contenir 0 ou plusieurs allergènes. Un allergène peut se trouver dans 0 ou plusieurs plats
plats → types_plat	N:1	Un plat appartient à exactement 1 type. Un type peut qualifier 0 ou plusieurs plats
menus → menu_images	1:N	Un menu peut avoir 0 ou plusieurs images. Une image appartient à exactement 1 menu
horaires → jours	1:1	Un horaire correspond à un jour. Un jour a exactement 1 horaire

c. Vues SQL

Afin de simplifier les requêtes récurrentes et d'éviter les jointures répétitives dans le code applicatif, plusieurs vues SQL ont été définies. Elles offrent une lecture agrégée et lisible des données sans modifier la structure des tables sous-jacentes.

Vue	Description
vue_menus_complets	Regroupe pour chaque menu ses thèmes, régimes alimentaires et images associées en colonnes agrégées
vue_plats_allergenes	affiche chaque plat avec la liste de ses allergènes concaténés
vue_menus_plats	Jointure entre les menus et leurs plats, utile pour l'affichage des compositions
vue_commandes_details	Vue principale des commandes enrichie des informations du menu commandé et du libellé du statut
vue_historique_commandes	Retrace l'évolution des statuts d'une commande avec l'identité du client et l'auteur de la modification

Ces vues sont utilisées en lecture seule et ne remplacent pas les tables de base. Elles sont particulièrement utiles pour les interfaces d'administration, le suivi des commandes et l'affichage des catalogues de menus.

4.2. Modèle NoSQL – collections MongoDB

En complément de la base de données relationnelle MySQL, une base MongoDB vite_gourmand_db est utilisée pour gérer des données à structure flexible ou nécessitant une synchronisation rapide. Elle contient trois collections.

a. Collection avis

Stocke les avis clients après une commande. Elle conserve une copie dénormalisée des informations client pour éviter des jointures côté NoSQL.

Champ	Type	Description
_id	ObjectId	Identifiant MongoDB
id_commande	Ref	Référence vers la commande MySQL
id_user	Ref	Référence vers l'utilisateur MySQL
numero_commande	String	Numéro de la commande
nom_client	String	Nom du client
prenom_client	String	Prénoms du client
note	Number	Note de 1 à 5
commentaire	String	Commentaire saisi par le client
statut	String	Statut de l'avis : en attente, approuvé et refusé
created_at	Date	Date de création
updated_at	Date	Date de mise à jour

b. Collection commandes

Champ	Type	Description
_id	ObjectId	Identifiant MongoDB
id_commande	Ref	Référence vers la commande MySQL
user_id	Ref	Référence vers l'utilisateur MySQL
menu_id	Ref	Référence vers le menu MySQL
numero_commande	String	Numéro de la commande
nom_client	String	Nom du client
prenom_client	String	Prénoms du client
email_client	String	Adresse email du client
gsm_client	String	Numéro de téléphone du client
adresse_prestation	String	Adresse de livraison de la commande
ville_prestation	String	ville de livraison de la commande

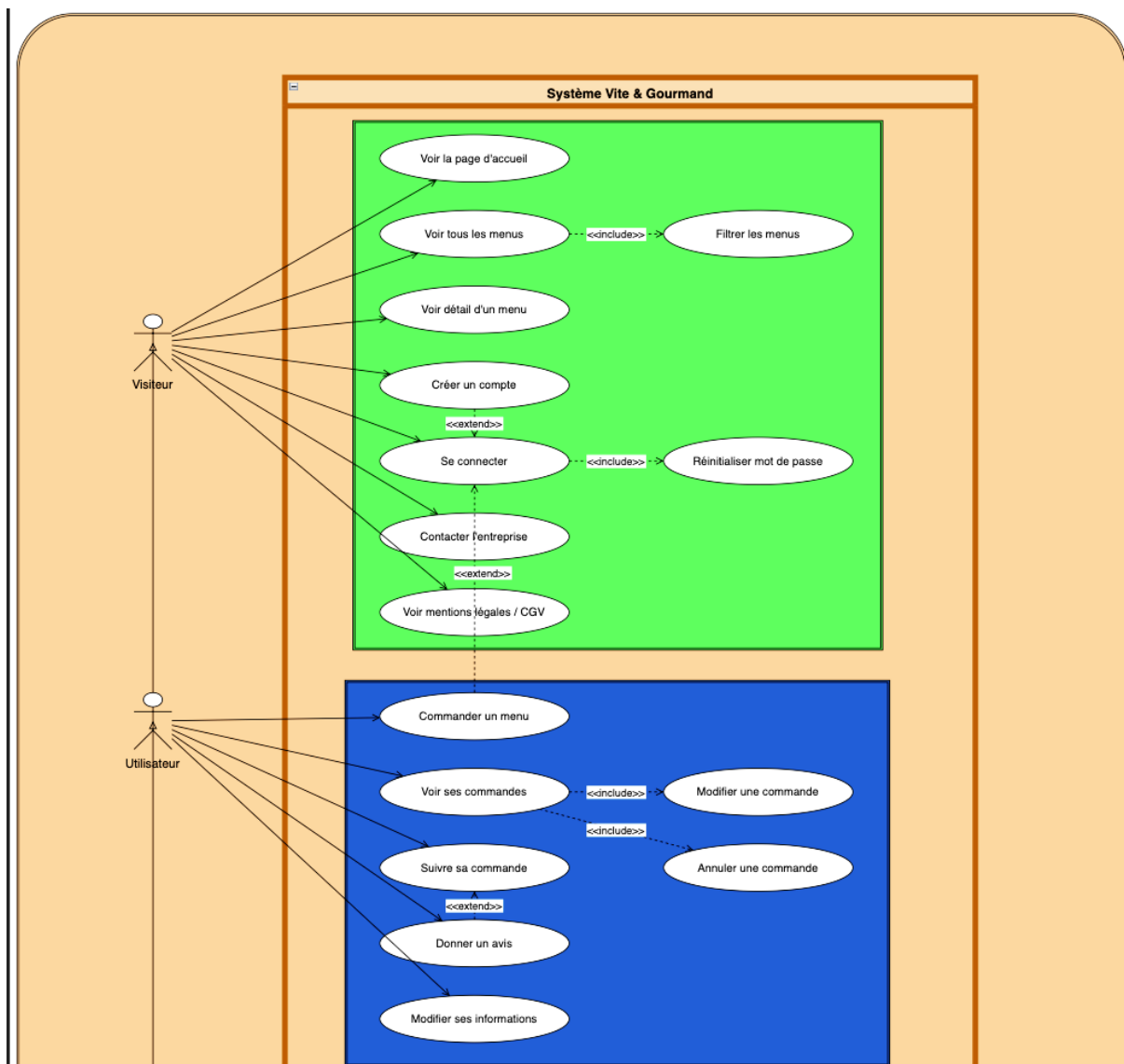
code_postal_orestation	String	Code postal de livraison de la commande
date_prestation	String	Date de livraison de la commande
heure_prestation	String	heure de livraison de la commande
nb_personnes	Number	Nombre de personnes choisi lors de la commande
menu_titre	String	Titre du menu commandé
prix_menu	Number	Prix du menu (prix de base fois le nombre de personnes choisi)
prix_livraison	Number	Frais de livraison si hors Bordeaux
prix_total	Number	Prix total de la commande
statut	String	Statut actuel de la commande
location_materiel	Boolean	Location de matériel (case cochée ou non lors de la commande)
mode_contact	String	Mode de contact du client
motif_annulation	String	Motif en cas d'annulation
commentaire	String	Commentaire du client lors de la commande
created_at	Date	Date de création
updated_at	Date	Date de mise à jour
synced_at	Date	Date de dernière synchronisation avec MySQL

c. Collection demandesClient

Champ	Type	Description
_id	ObjectId	Identifiant MongoDB
titre	String	Titre de la demande
description	String	Contenu de la demande
email	String	Email de contact du demandeur
created_at	Date	Date d'envoi de la demande

5. Diagrammes d'utilisation

5.1. Diagramme de cas d'utilisation (Use Case)



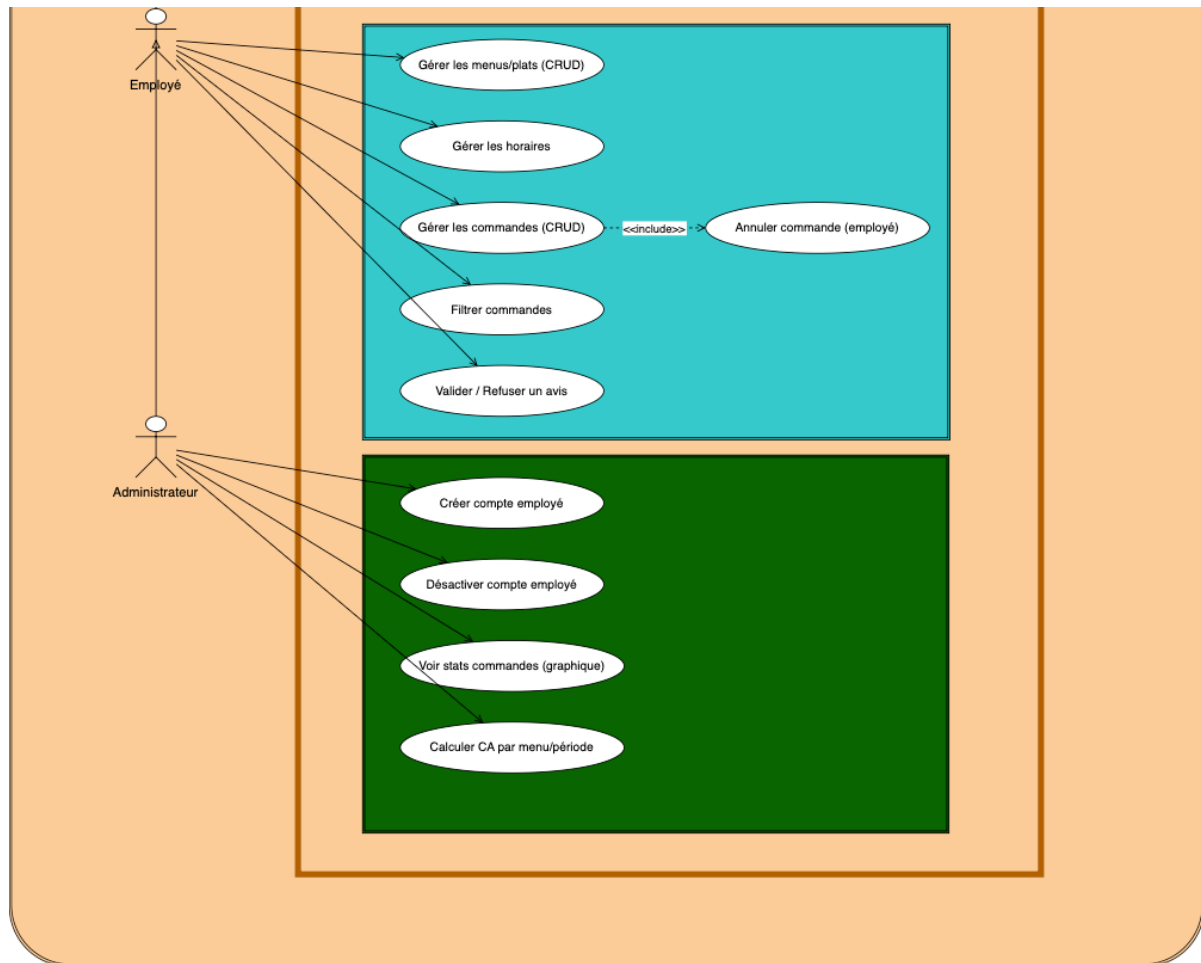


Figure 3 : Diagramme de cas d'utilisation

Visiteur peut :

- Consulter la page d'accueil
- Consulter les menus (avec filtres)
- Voir le détail d'un menu
- Créer un compte
- Se connecter
- Contacter l'entreprise

Utilisateur (connecté) peut :

- Tout ce que fait le visiteur
- Commander un menu
- Consulter ses commandes
- Modifier / annuler une commande (si non acceptée)
- Suivre sa commande (si acceptée)
- Laisser un avis (si commande terminée)
- Modifier ses informations personnelles

Employé peut :

- Tout ce que fait l'utilisateur
- Gérer les menus (CRUD)
- Gérer les plats et les horaires
- Gérer les commandes (CRUD)
- Valider ou refuser les avis clients

Administrateur peut :

- Tout ce que fait un employé
- Créer / désactiver des comptes employés
- Consulter les statistiques de commandes (graphiques MongoDB)
- Consulter le chiffre d'affaires avec filtres par menu et par période

5.2. Diagramme de séquence

a. Commande d'un menu

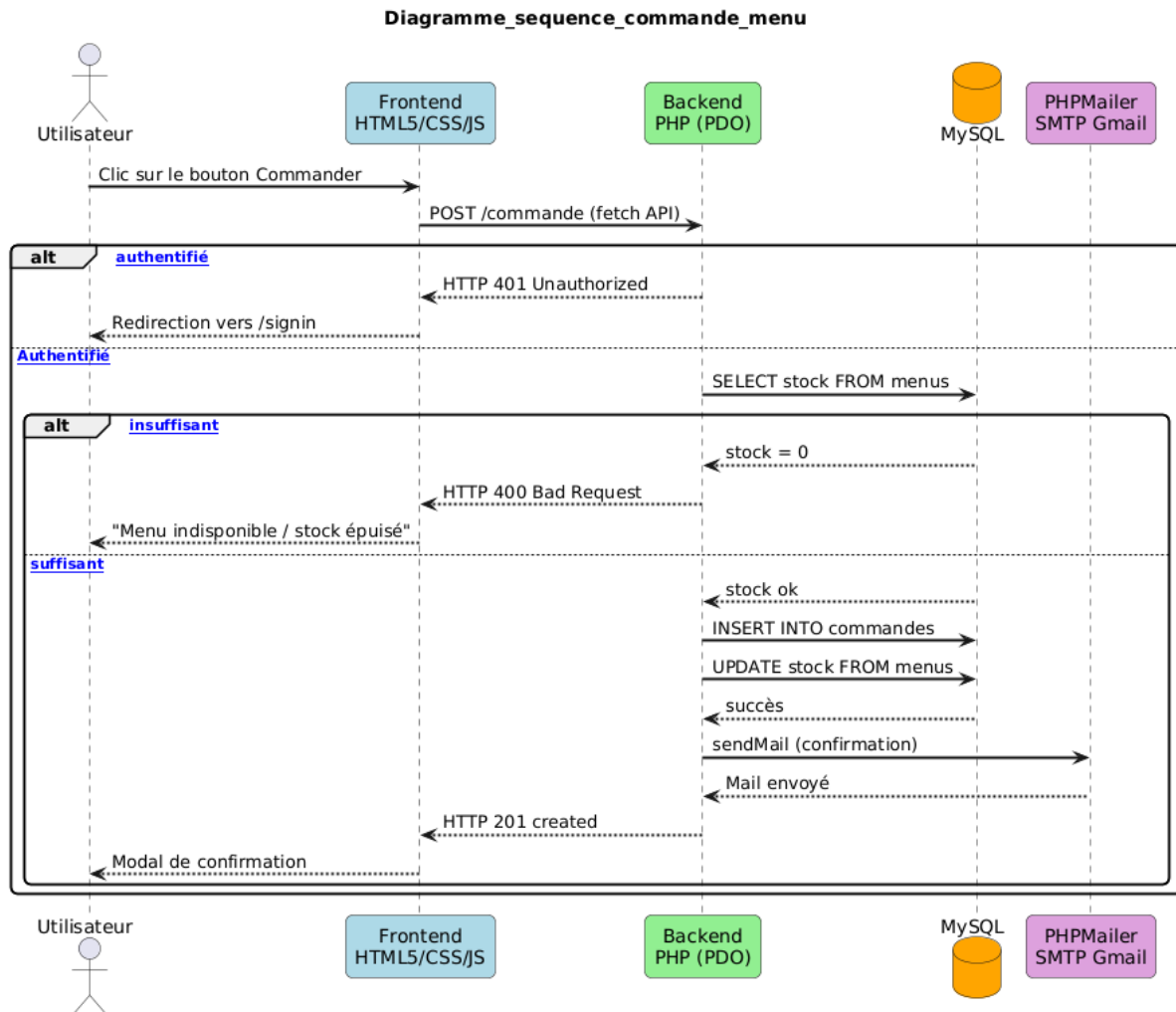


Figure 4 : Diagramme de séquence - Commande d'un menu

b. Commande terminée – Avis client

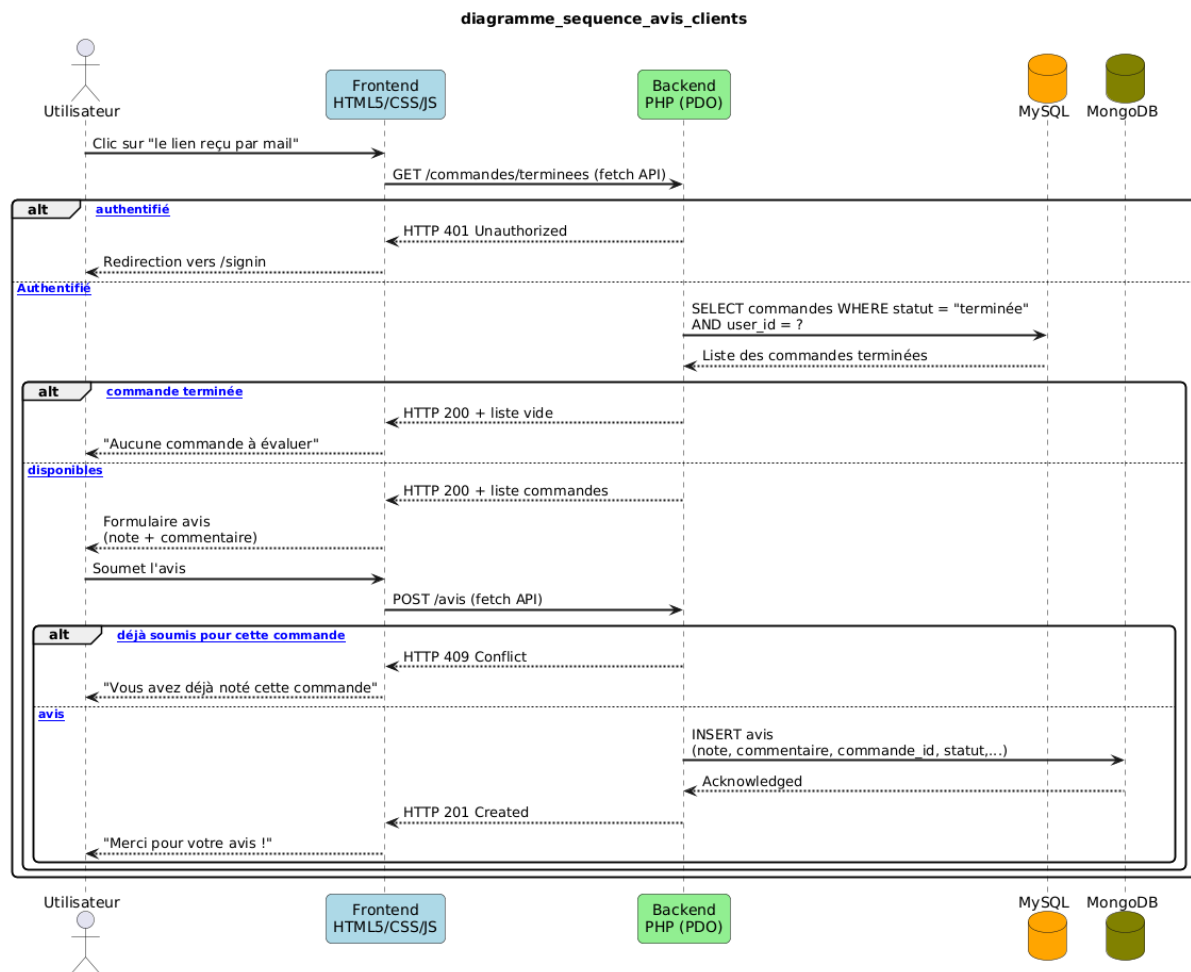


Figure 5 : [Diagramme de séquence – Avis client](#)

6. Déploiement de l'application

6.1. Prérequis

Avant de procéder au déploiement, les outils suivants doivent être installés et configurés :

- Git : Gestionnaire de versions utilisé pour pousser le code vers Heroku
- Heroku CLI : Interface en ligne de commande pour gérer l'application Heroku
- Un compte Heroku : Plateforme cloud utilisée pour l'hébergement
- Un compte MongoDB Atlas : Pour la base de données MongoDB en production
- Un addon JawsDB : Pour la base de données MySQL en production
- Un compte Cloudinary : Pour le stockage des images

6.2. Architecture de déploiement

L'application Vite Gourmand est déployée sur une seule application Heroku, avec un serveur Nginx configuré pour servir à la fois le frontend et le backend depuis un même dépôt Git.

- Frontend : Application HTML/CSS/JavaScript vanilla, servie depuis /app/frontend
- Backend : API REST en PHP, accessible via le préfixe /api/, traité par PHP-FPM
- Nginx : Serveur web configuré via config/nginx.conf.erb pour router les requêtes
- Composer : Gestionnaire de dépendances PHP, avec les vendors dans backend/vendor/

Les bases de données (MySQL et MongoDB) sont hébergées sur des services cloud externes (JawsDB et MongoDB Atlas). Les fichiers médias sont hébergés sur Cloudinary.

6.3. Structure du projet

L'organisation des fichiers est la suivante :

```
Projet_ecf_vitegourmand/  
--- Procfile  
--- composer.json  
--- config/  
    ---- nginx.conf.erb  
--- frontend/  
    ---- index.html  
    ---- js/  
    ---- scss/  
--- backend/  
    ---- public/  
        ---- index.php  
--- vendor/
```

6.4. Fichiers de configuration Heroku

a. Procfile

Le Procfile indique à Heroku comment démarrer l'application. Il spécifie l'utilisation de Nginx avec un fichier de configuration personnalisé et le dossier backend/public/ comme point d'entrée PHP :

```
web: backend/vendor/bin/heroku-php-nginx -C config/nginx.conf.erb  
backend/public/
```

b. config/nginx.conf.erb

Ce fichier configure le routing Nginx. Les requêtes vers /api/ sont transmises à PHP-FPM qui exécute backend/public/index.php. Toutes les

autres requêtes servent le frontend SPA depuis /app/frontend, avec un fallback vers index.html pour le routing côté client :

```
# Redirection des routes SPA (frontend)
location / {
    root /app/frontend;
    try_files $uri $uri/ /index.html;
}

# API PHP
location /api/ {
    try_files $uri /index.php$is_args$args;
    location ~ /\.php$ {
        include fastcgi_params;
        fastcgi_pass heroku-fcgi;
        fastcgi_param SCRIPT_FILENAME /app/backend/public/index.php;
    }
}
```

c. composer.json

Le fichier composer.json définit les dépendances PHP du projet. Le paramètre vendor-dir est configuré sur backend/vendor afin que Heroku installe les dépendances au bon emplacement, cohérent avec le Procfile :

```
{ "require": {
    "ext-mongodb": "*",
    "phpmailer/phpmailer": "^7.0",
    "vlucas/phpdotenv": "^5.6",
    "mongodb/mongodb": "^2.2",
    "cloudinary/cloudinary_php": "^3.1"
},
"config": {
    "vendor-dir": "backend/vendor"
}
}
```

6.5. Déploiement

a. Étape 1 - Création de l'application Heroku

- heroku login
- heroku create nom_application (ici : vitegourmand-ecf2026)

b. Étape 2 - Configuration des variables d'environnement

Les variables sensibles sont configurées directement sur Heroku et ne sont jamais stockées dans le code. La configuration peut être faite de deux façons :

- En ligne de commande :
 - `heroku config:set DB_HOST=votre-host-jawsdb`
 - `heroku config:set DB_NAME=votre-db`
 - `heroku config:set DB_USER=votre-user`
 - `heroku config:set DB_PASS=votre-password`
 - `heroku config:set MONGODB_URI=mongodb+srv://...`
 - `heroku config:set JWT_SECRET=votre-secret`
 - `heroku config:set CLOUDINARY_URL=cloudinary://...`
 - ...
- Dashboard > vitegourmand-ecf2026 > Settings > Config Vars > Reveal Config Vars

c. Étape 3 - Déploiement

- `git push heroku main`

6.6. Base de données

a. MySQL (JawsDB)

JawsDB est un add-on Heroku fournissant une base de données MySQL hébergée dans le cloud.

- Ajout de l'add-on JawsDB depuis le dashboard Heroku
- Récupération des credentials de connexion
- Import du schéma SQL via :
 - `mysql -h hostname -u username -ppassword database < backup.sql`
- Configuration des variables d'environnement sur Heroku

b. MongoDB Atlas

MongoDB Atlas est utilisé pour stocker les données non relationnelles.

- Création d'un cluster gratuit sur MongoDB Atlas
- Création d'un utilisateur avec les droits lecture/écriture
- Whitelist de l'IP Heroku (0.0.0.0/0 pour autoriser toutes les IPs)
- Récupération de l'URI de connexion
- Configuration de la variable MONGODB_URI sur Heroku

c. Cloudinary

Cloudinary est utilisé pour stocker les images des menus.

- Création des clés api depuis le menu dashboard du compte cloudinary
- Configuration des variables cloudinary sur Heroku

6.7. Vite-et-gourmand.fr, DSN et sécurité

Le domaine vite-et-gourmand.fr a été acheté sur OVH. Ils constitue l'adresse publique de l'application en production.

a. Délégation DNS vers Cloudflare

Afin de bénéficier des services de Cloudflare (SSL, CDN, protection), les nameservers OVH ont été remplacés par les nameservers Cloudflare depuis l'interface OVH.

[← Retour à la liste de services](#)Informations générales Zone DNS **Serveurs DNS** Redirection DynHost Hosts DS Records Gestion des contacts Pour vous assurer du bon fonctionnement de votre configuration, vous pouvez utiliser [cet outil de vérification DNS](#). **Modifier les DNS****Résilier DNS Anycast**

Serveurs DNS	IP associée	Statut	Type de serveurs DNS
clara.ns.cloudflare.com		Actif	Personnalisé
merlin.ns.cloudflare.com		Actif	Personnalisé

Figure 6 : Délégation DNS vers Cloudflare

Cloudflare gère désormais l'intégralité des enregistrements DNS du domaine.

b. Configuration DNS

Type	Nom	Contenu	Statut proxy
Cname	vite-et-gourmand.fr	convex-iris-qt5g6ap16ootckonwvklfd6x.herokudns.com	Proxied
Cname	www	arcane-marlin-nwkez1p05apwbo874km0089f.herokudns.com	Proxied

Les deux enregistrements pointent vers l'application déployée sur Heroku.

c. Configuration Cloudflare

Paramètre	Valeur	Rôle
Mode SSL/TLS	Full	Chiffrement bout en bout
Always HTTPS	Activé	Redirection HTTP → HTTPS automatique
Cache level	Standard	Mise en cache des assets statiques
Nameservers	Délégués OVH → Cloudflare	Gestion DNS centralisée

d. DNSSEC :

- Activé via Cloudflare (DNS → Settings → DNSSEC)
- Enregistrement DS ajouté sur OVH
- Key Tag : 2371
- Algorithme : 13 – ECDSA256SHA256
- Statut : Actif

Cloudflare agissant comme proxy, l'IP visible par Heroku est celle de Cloudflare et non celle de l'utilisateur final. Aucune configuration supplémentaire n'a été nécessaire côté application.

6.8. Variables d'environnement

Key	value
APP_DEBUG	false
APP_ENV	production
APP_SECRET	*****
BD_HOST	*****
CLOUDINARY_API_KEY	*****
CLOUDINARY_API_SECRET	*****
CLOUDINARY_CLOUD_NAME	*****
CONTACT_PHONE	“** ** ** ** **”
DB_NAME	*****
DB_PASSWORD	*****
DB_USER	*****
FRONTEND_URL	vite-et-gourmand.fr
JAWSDB_URL	mysql://...
MAIL_FROM_ADDRESS	vitegourmandecf26@gmail.com
MAIL_FROM_NAME	"Vite & Gourmand"
MAIL_HOST	smtp.gmail.com
MAIL_PASSWORD	*****
MAIL_PORT	587
MAIL_USERNAME	vitegourmandecf26@gmail.com
MONGO_DB	*****
MONGO_URI	mongodb+srv://...
MYSQL_PORT	3306

6.9. Vérification du déploiement

Après le déploiement, les vérifications suivantes ont été effectuées :

- ⇒ Accès à l'URL de l'application
- ⇒ Connexion à l'API backend via /api/
- ⇒ Authentification (JWT)
- ⇒ Lecture/écriture en base de données MySQL
- ⇒ Données enregistrées dans MongoDB
- ⇒ Upload et affichage des images via Cloudinary
- ⇒ Toutes les routes protégées fonctionnent correctement

6.10. Problèmes rencontrés et solutions

Problème	Solution
Vendors PHP introuvable au démarrage	Paramètre vendor-dir dans composer.json pour pointer vers backend/vendor
Absence d'un composer.json et d'un composer.lock au 1 ^{er} push	Ajout de ces deux fichiers à la racine du projet avec la configuration décrite dans 5.4 - c

CORS bloqué entre frontend et backend	Configuration du header <code>ALLOWED_ORIGIN</code> dans <code>index.php</code>
Connexion MongoDB refusée	Whitelist des IPs sur MongoDB Atlas (0.0.0.0/0)
Routing SPA ne fonctionnait pas en production	Configuration Nginx avec <code>try_files</code> vers <code>index.html</code> pour les routes frontend
Routes <code>/api/</code> retournaient 404	Ajout d'un bloc de location <code>/api/</code> dans <code>nginx.conf.erb</code> avec <code>fastcgi_pass</code> vers <code>heroku-fcgi</code>